

Olist E-Commerce Analytics

dbt + BigQuery Portfolio Project

Soichiro Tanabe

April 2026

Table of Contents

1. Executive Summary
2. Business Scenario
3. Objectives
4. Dataset
5. Architecture Overview
6. Technologies Used
7. Project Structure
8. Data Pipeline Workflow
9. Testing & Documentation
10. Key Learnings
11. Challenges and Solutions
12. Future Improvements
13. Project Completion Summary
14. References

1. Executive Summary

This project presents the design and implementation of a data transformation pipeline using dbt (data build tool) and Google BigQuery to analyse the Brazilian E-Commerce dataset published by Olist. The primary objective is to transform raw e-commerce data into analytics-ready models following industry best practices, demonstrating core analytics engineering skills.

The solution is built using dbt Cloud for transformation orchestration and Google BigQuery as the cloud data warehouse. The pipeline follows a layered architecture, where raw data is first cleaned in staging models and then joined and aggregated into business-ready mart models. The project includes 6 staging models, 4 mart models, 23 automated data quality tests, and full documentation.

The project demonstrates practical skills in analytics engineering, including dimensional modelling, SQL transformation, data quality testing, documentation, and deployment via GitHub.

2. Business Scenario

Olist is a Brazilian e-commerce platform that connects merchants with customers across multiple marketplaces. The raw dataset contains transactional data spread across 9 separate tables, including orders, customers, products, sellers, payments, and reviews.

In a real business setting, analysts need clean, joined, business-ready tables to answer questions such as:

- Which product categories generate the most revenue?
- Which sellers have the best and worst review scores?
- How do delivery times affect customer review scores?
- Where are our top customers and sellers located?

Raw data alone cannot answer these questions efficiently. This project transforms the raw Olist data into a structured analytical layer that supports these business queries directly.

3. Objectives

The primary objectives of this project are:

- Connect Google BigQuery to dbt Cloud as the development environment
- Load the Olist dataset into BigQuery as raw source tables
- Build staging models that clean and standardise each source table
- Build mart models that join tables and apply business logic
- Implement automated data quality tests across all models
- Document all models and columns using schema.yml
- Push the complete project to GitHub as a portfolio artifact

4. Dataset

4.1 Source

The dataset used in this project is the Brazilian E-Commerce Public Dataset by Olist, available on Kaggle.

Source: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce>

License: Creative Commons CC BY-NC-SA 4.0
(<https://creativecommons.org/licenses/by-nc-sa/4.0/>)

This license permits free use for non-commercial purposes, provided the original source is credited.

4.2 Raw Tables

The dataset was loaded into Google BigQuery under the raw_olist dataset, containing 9 tables:

Table Name	Description	Key Columns
olist_orders_dataset	Order header information	order_id, customer_id, order_status
olist_customers_dataset	Customer information	customer_id, customer_city, customer_state
olist_order_items_dataset	Line items per order	order_id, product_id, seller_id, price
olist_order_payments_dataset	Payment information	order_id, payment_type, payment_value
olist_order_reviews_dataset	Customer reviews	review_id, order_id, review_score
olist_products_dataset	Product catalogue	product_id, product_category_name
olist_sellers_dataset	Seller information	seller_id, seller_city, seller_state
olist_geolocation_dataset	Zip code geo data	geolocation_zip_code_prefix
product_category_name_translation	Portuguese to English	category_name, category_name_english

5. Architecture Overview

The project follows the standard dbt layered architecture pattern:

Layer	Purpose	Models
Sources (raw_olist)	Raw data loaded from Kaggle CSVs into BigQuery	9 source tables
Staging (stg_*)	Clean and rename raw tables 1:1, no joins	6 staging models
Marts (fct_* / dim_*)	Joined, business-ready tables with logic	4 mart models

Data flows as follows:

```
rawolist (BigQuery source tables)
  |
  v
Staging models (stg_*) -- clean, rename, type-cast
  |
  v
Mart models (fct_orders, dim_customers, dim_products, dim_sellers)
  |
  v
dbtolist dataset in BigQuery (analytics-ready tables)
```

6. Technologies Used

Technology	Purpose	Details
dbt Cloud	Transformation orchestration	Runs SQL models, tests, documentation
Google BigQuery	Cloud data warehouse	Stores raw and transformed data
GitHub	Version control	Hosts project code publicly for portfolio
SQL	Data transformation language	Used for all staging and mart models
YAML	Configuration and documentation	sources.yml, schema.yml, dbt_project.yml

6.1 Technology Selection and Justification

dbt Cloud was selected as the transformation tool for this project due to its rapid adoption across the industry. As of Q4 2024, dbt appears in 62% of data engineering job postings, the fastest year-on-year growth of any tool in the modern data stack (Reliable Data Engineering, 2026). It has become the backbone of modern analytics engineering (Likki, 2026), and its native integration with Google BigQuery enables teams to automate complex data transformations efficiently within a single, cohesive workflow (dbt Labs, n.d.).

Google BigQuery was selected for its native integration with dbt Cloud and its accessible pricing model, offering 10GB free storage and 1TB of free queries per month

(Google Cloud, n.d.). BigQuery uses standard SQL syntax, making transformations readable, portable, and straightforward to maintain.

GitHub was used for version control to make the project publicly viewable for a portfolio project.

7. Project Structure

7.1 File Structure

```
olist_portfolio/
├── models/
│   ├── staging/
│   │   ├── sources.yml
│   │   ├── schema.yml
│   │   ├── stg_orders.sql
│   │   ├── stg_customers.sql
│   │   ├── stg_order_items.sql
│   │   ├── stg_order_payments.sql
│   │   ├── stg_order_reviews.sql
│   │   ├── stg_products.sql
│   │   └── stg_sellers.sql
│   └── marts/
│       ├── schema.yml
│       ├── fct_orders.sql
│       ├── dim_customers.sql
│       ├── dim_products.sql
│       └── dim_sellers.sql
├── dbt_project.yml
└── README.md
```

7.2 Staging Models

Staging models clean and standardise each raw source table on a 1:1 basis. No joins are performed at this layer. Each model selects specific columns, applies consistent naming, and references the source using dbt's `source()` macro.

Model	Source Table	Key Transformations
stg_orders	olist_orders_dataset	Select key columns, rename for clarity
stg_customers	olist_customers_dataset	Clean customer location data

stg_order_items	olist_order_items_dataset	Select price, freight, product/seller IDs
stg_order_payments	olist_order_payments_dataset	Select payment type and value
stg_order_reviews	olist_order_reviews_dataset	Select review score and comments
stg_products	olist_products_dataset	Select product attributes
stg_sellers	olist_sellers_dataset	Clean seller location data

7.3 Mart Models

Mart models join staging models together and apply business logic to create analytics-ready tables. The project follows dimensional modelling principles with one fact table and three dimension tables.

Model	Type	Joins	Business Question Answered
fct_orders	Fact	stg_orders + stg_order_items + stg_order_payments	How much revenue per order?
dim_customers	Dimension	stg_customers + fct_orders	Where are customers? How much do they spend?
dim_products	Dimension	stg_products + product_category_translation	What are products called in English?
dim_sellers	Dimension	stg_sellers + stg_order_reviews + stg_order_items	Which sellers have best review scores?

8. Data Pipeline Workflow

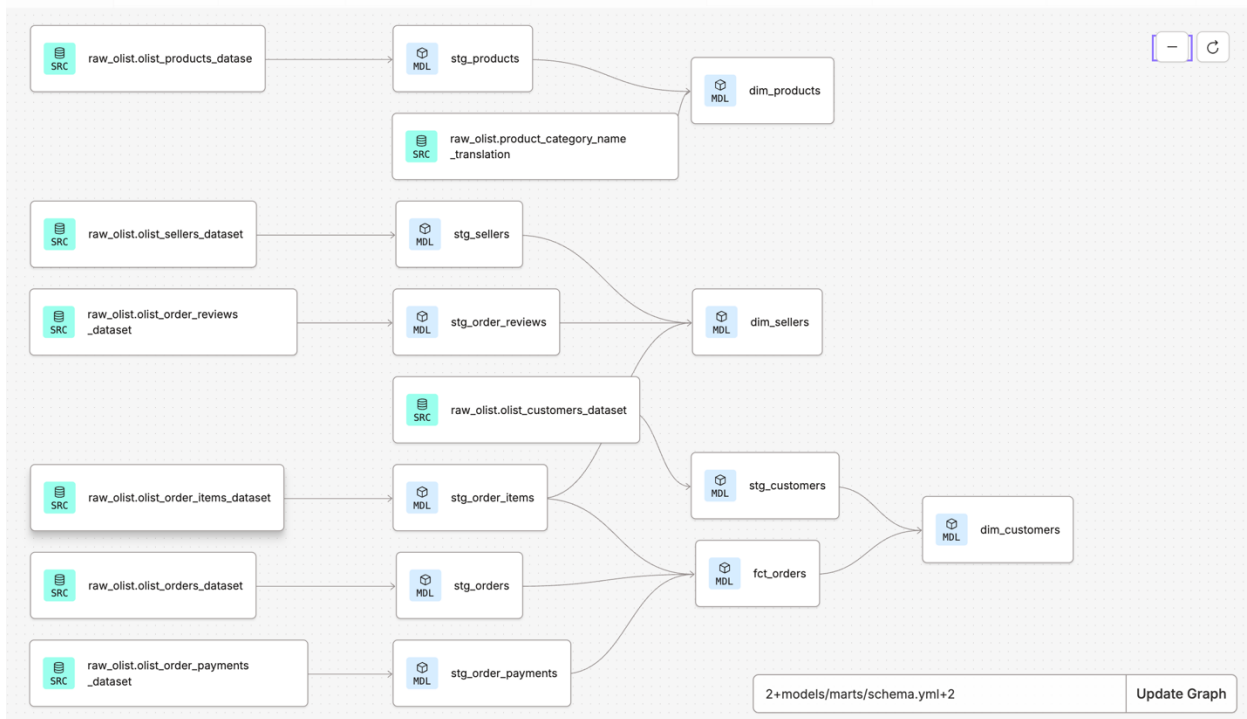


Figure 1: Workflow

8.1 Source Configuration

All raw tables are defined in `sources.yml`, which tells dbt where to find the raw data in BigQuery. This enables the use of the `source()` macro in staging models, creating traceable data lineage.

8.2 Staging Layer

Each staging model follows the same pattern: a source CTE that references the raw table using `source()`, a renamed CTE that selects and renames columns, and a final `SELECT`. This pattern ensures consistency across all models.

8.3 Mart Layer - `fct_orders`

The orders fact table joins three staging models to calculate revenue metrics per order. Key business logic applied:

- Total payment value summed across all payment methods per order
- Total items counted per order
- Total freight value calculated per order

- Orders joined with items and payments using order_id

8.4 Mart Layer - dim_customers

The customer dimension enriches customer data with order statistics. Key business logic applied:

- Separate counts for delivered vs cancelled orders
- Total spend calculated only for delivered orders (not cancelled)
- COALESCE used to handle customers with no orders (return 0 instead of NULL)

8.5 Mart Layer - dim_sellers

The seller dimension calculates performance metrics per seller:

- Total orders fulfilled per seller
- Average review score calculated by joining reviews through order items
- COALESCE used for sellers with no reviews

8.6 Mart Layer - dim_products

The product dimension translates Portuguese category names to English:

- LEFT JOIN with product_category_name_translation table
- Preserves all products even if translation is missing
- Includes physical dimensions for logistics analysis

8.7 Materialisation Strategy

All models are materialised as tables (not views) in BigQuery. This is configured globally in dbt_project.yml:

```
models:
  my_new_project:
    staging:
      +materialized: table
    marts:
      +materialized: table
```

Tables were chosen over views due to BigQuery's cross-region query requirements — materialising as tables ensures all data is stored in the EU region consistently.

9. Testing & Documentation

9.1 Data Quality Tests

23 automated data quality tests were implemented across all staging and mart models using dbt's built-in testing framework. Tests are defined in schema.yml files.

Test Type	Description	Applied To
unique	Ensures no duplicate values in a column	All primary key columns
not_null	Ensures no NULL values in a column	All key and required columns

Example of a known data quality issue discovered: review_id in the order reviews table is not unique — some reviews appear multiple times. This reflects a real data quality issue in the source dataset, and the unique test was removed for this column with documentation of the finding.

9.2 Test Results

Final test run results:

Metric	Result
Total tests	23
Passed	23
Failed	0
Warnings	0

9.3 Documentation

All models and key columns are documented in schema.yml files. Documentation includes:

- Model-level descriptions explaining the purpose of each model
- Column-level descriptions for key columns
- Test definitions attached to documented columns

dbt docs generate can be run to produce a browsable documentation website with a full lineage graph showing how all models connect.

10. Key Learnings

- Gained hands-on experience with the full dbt project lifecycle: sources, staging, marts, tests, documentation, and deployment
- Learned how to configure dbt Cloud with BigQuery including service account authentication
- Understood the importance of dataset region configuration in BigQuery — mismatched EU/US regions caused cross-region query failures that required careful debugging
- Applied dimensional modelling principles (fact and dimension tables) to a real e-commerce dataset
- Discovered a real data quality issue (duplicate review_id values) and handled it appropriately
- Understood the difference between views and tables in BigQuery and when to use each materialisation strategy
- Practised version control with GitHub including branching, pull requests, and merging

11. Challenges and Solutions

11.1 BigQuery Region Mismatch

The raw_olist dataset was created in the EU region, but dbt was initially trying to write models to a dbt_olist dataset that was in the US region. This caused 404 errors when dbt tried to query cross-region.

Solution: Deleted and recreated the dbt_olist dataset in the EU region, and set the Location field to EU in the dbt Cloud BigQuery connection optional settings. Also changed model materialisation from views to tables to ensure physical data was stored in the correct region.

11.2 CSV Upload Errors for Reviews Table

The olist_order_reviews_dataset CSV contained customer review comments with special characters, quotes, and newlines that caused BigQuery's CSV parser to fail with 100+ errors.

Solution: Enabled the Quoted newlines option in BigQuery's advanced CSV upload settings and set the Maximum bad records threshold to 200, allowing the table to load successfully despite the malformed rows.

11.3 Wrong CSV Uploaded for Orders Table

During the initial upload of 9 CSV files, the customers CSV was accidentally uploaded under the olist_orders_dataset table name. This caused a column not found error when running the orders staging model.

Solution: Deleted the incorrectly loaded table, verified the correct CSV file by checking its header columns locally, and re-uploaded the correct file.

11.4 dbt Cloud Connection Switching

The existing dbt Cloud account was connected to Snowflake from the dbt Fundamentals course. Switching the project connection to BigQuery through the UI was complex, as the credentials panel did not allow direct connection type changes.

Solution: Created a separate dbt Cloud account using a different email address, allowing a clean BigQuery connection setup from scratch without affecting the existing Snowflake-based course project.

12. Future Improvements

12.1 Additional Mart Models

The project could be extended with additional mart models such as:

- fct_reviews — aggregated review analysis with delivery time correlation
- dim_geolocation — geographic lookup table for mapping customer and seller locations

12.2 Advanced dbt Features

As additional dbt Milestones are completed, the following features will be incorporated:

- Jinja macros for reusable SQL patterns (Milestone 2)
- Incremental models to efficiently process new data (Milestone 3)
- dbt Mesh for modular project architecture (Milestone 3)

- Advanced testing with custom generic tests (Milestone 3)

12.3 Visualisation Layer

The mart models in BigQuery could be connected to a BI tool such as Looker Studio or Tableau to create interactive dashboards answering the business questions outlined in Section 2.

12.4 CI/CD Pipeline

A continuous integration pipeline could be configured in dbt Cloud to automatically run dbt test on every pull request, ensuring data quality is maintained as the project evolves.

13. Project Completion Summary

The following milestones were completed for this project:

- ✓ Set up Google BigQuery project (dbt-portfolio)
- ✓ Loaded Olist dataset — 9 tables into raw_olist dataset in EU region
- ✓ Created dbt Cloud account and connected to BigQuery
- ✓ Configured GitHub repository (olist_portfolio) for version control
- ✓ Built 6 staging models (stg_orders, stg_customers, stg_order_items, stg_order_payments, stg_order_reviews, stg_products, stg_sellers)
- ✓ Built 4 mart models (fct_orders, dim_customers, dim_products, dim_sellers)
- ✓ Configured global table materialisation in dbt_project.yml
- ✓ Wrote 23 passing data quality tests (unique, not_null)
- ✓ Documented all models with descriptions in schema.yml
- ✓ Pushed complete project to GitHub via pull request

14. References

dbt Labs (n.d.) *Transform data at scale with dbt + BigQuery*. Available at: <https://www.getdbt.com/data-platforms/bigquery> (Accessed: April 2026).

Olist (n.d.) Brazilian E-Commerce Public Dataset. Available at: <https://www.kaggle.com/datasets/olistbr/brazilian-ecommerce> (Accessed: April 2026).

Creative Commons (n.d.) CC BY-NC-SA 4.0 License. Available at: <https://creativecommons.org/licenses/by-nc-sa/4.0/> (Accessed: April 2026).

Google Cloud (n.d.) BigQuery Documentation. Available at: <https://cloud.google.com/bigquery/docs> (Accessed: April 2026).

Google Cloud (n.d.b) *BigQuery pricing*. Available at: <https://cloud.google.com/bigquery/pricing> (Accessed: April 2026).

Likki, L. (2026) 'How I Install dbt: A Step-by-Step Guide (Local + Cloud)', *Medium*, 9 April. Available at: <https://medium.com/@likkilaxminarayana/5-installing-dbt-step-by-step-local-cloud-60cf7d5579b7> (Accessed: April 2026).

Reliable Data Engineering (2026) 'The Modern Data Stack in 2025: What Actually Won', *Medium*, 3 January. Available at: <https://medium.com/@reliabledataengineering/the-modern-data-stack-in-2025-what-actually-won-708c59176b32> (Accessed: April 2026).